Stochastics & Machine Learning

Theory Recap

252-0836-00L · D-MAVT, ETH Zürich

Carlos Cotrini, Felix Friedrich, Andreas Streich · Spring 2026 (FS2026)

This recap follows the structure of the lecture notes (*script*) and slide decks, condensed to what is needed for the exam: definitions, key formulas, and worked recipes, with derivations trimmed to short sketches. **Key Result** boxes collect the formulas you are most likely to need directly; **Exam Tip** boxes flag common pitfalls and the way concepts tend to be examined. The final section is a compact *Exam Quick Reference* with all formulas and distribution tables in one place.

Course map. The course has two halves. **Part I (Stochastics)** builds probability theory from the ground up (Ch. 1–5), then statistics (Ch. 6–12): laws of large numbers, descriptive statistics, estimation, hypothesis testing and confidence intervals. **Part II (Machine Learning)** covers supervised learning (regression, trees, ensembles), the Bayesian view of learning and Gaussian Processes, deep learning (neural networks, CNNs, NLP/Transformers), unsupervised learning (clustering, dimensionality reduction), generative models (autoencoders, GANs, diffusion models), and reinforcement learning.

Generated theory recap — compile date June 13, 2026

Contents

Part I — Probability Foundations

1 Probability (Ch. 1)

1.1 Random Experiments, Events and Sets

A *random experiment* has a sample space Ω (set of all possible outcomes). An *event* $A \subseteq \Omega$ is a subset of outcomes. Standard set operations apply: union $A \cup B$ ("or"), intersection $A \cap B$ ("and"), complement A^c ("not"), and A, B are *disjoint* if $A \cap B = \emptyset$.

A probability measure P on Ω satisfies:

1. $P(A) \geq 0$ for all events A;
2. $P(\Omega) = 1$;
3. For pairwise disjoint $A_1, A_2, \ldots$: $P\left(\bigcup_i A_i\right) = \sum_i P(A_i)$ (countable additivity).

Consequences: $P(A^c) = 1 - P(A)$, $P(\emptyset) = 0$, and the inclusion–exclusion formula $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

1.1.1 Laplace Experiments

If Ω is finite and all outcomes are equally likely,

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\#\text{favorable outcomes}}{\#\text{possible outcomes}}.$$

This is the bridge to combinatorics (Ch. 2): computing $P(A)$ reduces to counting $|A|$ and $|\Omega|$.

1.1.2 Continuous Models

For continuous sample spaces (e.g. $\Omega = [0, 1]$), probability is modeled via a density and $P(A) = \int_A f(x)\, dx$; single points have probability zero. This is developed fully in Ch. 4.

1.2 Conditional Probability and Bayes' Theorem

For $P(B) > 0$,

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Rearranged: $P(A \cap B) = P(A \mid B)\, P(B)$ (multiplication rule / chain rule for several events: $P(A_1 \cap \cdots \cap A_n) = P(A_1) P(A_2 \mid A_1) \cdots P(A_n \mid A_1 \cap \cdots \cap A_{n-1})$).

Probability trees represent sequential experiments: multiply probabilities along branches to get joint probabilities, and sum over branches leading to the same outcome.

If $B_1, \ldots, B_k$ partition Ω (pairwise disjoint, $\bigcup_i B_i = \Omega$), then for any event A:

$$P(A) = \sum_{i=1}^{k} P(A \mid B_i)\, P(B_i).$$

For $P(A), P(B) > 0$:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}.$$

Combined with the law of total probability (with $B_1, \dots, B_k$ partitioning Ω):

$$P(B_i \mid A) = \frac{P(A \mid B_i)\, P(B_i)}{\sum_{j=1}^{k} P(A \mid B_j)\, P(B_j)}.$$

Terminology: $P(B_i)$ = *prior*, $P(A \mid B_i)$ = *likelihood*, $P(B_i \mid A)$ = *posterior*.

"Low rate fallacy" / base-rate problems (e.g. rare disease testing): always set up the full partition (disease / no disease) with priors *and* both conditional probabilities (sensitivity $P(+ \mid \text{disease})$ and false-positive rate $P(+ \mid \text{no disease})$) before applying Bayes. A high sensitivity test can still have $P(\text{disease} \mid +)$ far below 1 if the disease is rare.

1.3 Independence

A and B are independent ($A \perp\!\!\!\perp B$) if

$$P(A \cap B) = P(A)\, P(B) \quad \Longleftrightarrow \quad P(A \mid B) = P(A) \;\; (\text{when } P(B) > 0).$$

A collection $A_1, \dots, A_n$ is (mutually) independent if for every subset $I \subseteq \{1, \dots, n\}$, $P\big(\bigcap_{i \in I} A_i\big) = \prod_{i \in I} P(A_i)$. Pairwise independence does *not* imply mutual independence.

A and B are conditionally independent given C (written $A \perp\!\!\!\perp B \mid C$) if $P(A \cap B \mid C) = P(A \mid C)\, P(B \mid C)$. Independence does not imply conditional independence and vice versa (Simpson-type paradoxes).

2 Combinatorics (Ch. 2)

Combinatorics counts outcomes for Laplace experiments ($P(A) = |A|/|\Omega|$). The key distinctions are *ordered vs. unordered* and *with vs. without replacement*.

Drawing k elements from a set of n distinguishable elements:

	Ordered	**Unordered**
Without replacement	$\dfrac{n!}{(n-k)!}$ (permutations)	$\binom{n}{k} = \dfrac{n!}{k!(n-k)!}$
With replacement	n^k	$\binom{n+k-1}{k}$

Special case $k = n$ without replacement, ordered: $n!$ permutations of all elements.

The number of ways to partition n distinguishable objects into groups of sizes $k_1, \ldots, k_r$ (with $\sum_i k_i = n$) is

$$\binom{n}{k_1, k_2, \ldots, k_r} = \frac{n!}{k_1!\, k_2! \cdots k_r!}.$$

This generalizes the binomial coefficient ($r = 2$) and underlies the Binomial/Multinomial distributions (Ch. 3).

Drawing colored balls. For an urn with several colors, probabilities of color compositions in a sample of size k are computed via the hypergeometric-type counting: $\binom{\text{color counts}}{\text{chosen}}$ combinations over $\binom{n}{k}$ total, generalizing to multiple colors via the multivariate hypergeometric distribution.

To decide which of the four formulas applies, ask: (1) *Does order matter?* (is "red then blue" different from "blue then red"?) and (2) *Can the same element be drawn twice?* (sampling with/without replacement, e.g. lottery numbers vs. dice rolls).

3 Random Variables (Ch. 3)

3.1 Discrete Random Variables

A discrete random variable (RV) X takes values in a countable set. Its **probability mass function (PMF)** is $p_X(x) = P(X = x)$, with $\sum_x p_X(x) = 1$. Its **cumulative distribution function (CDF)** is

$$F_X(x) = P(X \le x) = \sum_{y \le x} p_X(y),$$

non-decreasing, right-continuous, $F_X(-\infty) = 0$, $F_X(\infty) = 1$.

3.2 Common Discrete Distributions

Distribution	**PMF** $p(x)$	**Support**	$\mathbb{E}[X]$	$\mathrm{Var}(X)$
$\mathrm{Unif}\{1,\dots,n\}$	$\frac{1}{n}$	$1,\dots,n$	$\frac{n+1}{2}$	$\frac{n^2-1}{12}$
$\mathrm{Bern}(p)$	$p^x(1-p)^{1-x}$	$\{0,1\}$	p	$p(1-p)$
$\mathrm{Bin}(n,p)$	$\binom{n}{x}p^x(1-p)^{n-x}$	$0,\dots,n$	np	$np(1-p)$
$\mathrm{Geom}(p)$	$(1-p)^{x-1}p$	$1,2,\dots$	$\frac{1}{p}$	$\frac{1-p}{p^2}$
$\mathrm{NegBin}(r,p)$	$\binom{x-1}{r-1}p^r(1-p)^{x-r}$	$r, r+1,\dots$	$\frac{r}{p}$	$\frac{r(1-p)}{p^2}$
$\mathrm{Poi}(\lambda)$	$e^{-\lambda}\frac{\lambda^x}{x!}$	$0,1,2,\dots$	λ	λ

Hypergeometric(N,K,n): drawing n without replacement from N items (K "successes"): $p(x) = \binom{K}{x}\binom{N-K}{n-x}/\binom{N}{n}$, $\mathbb{E}[X] = n\frac{K}{N}$.

Binomial vs. Geometric vs. Negative Binomial all count Bernoulli(p) trials but ask different questions: Binomial = #successes in n fixed trials; Geometric = #trials until the *first* success; Negative Binomial = #trials until the r-th success. **Poisson** arises as the limit of $\mathrm{Bin}(n,p)$ as $n \to \infty$, $p \to 0$, $np \to \lambda$ — used for rare events / counts per unit time.

3.3 Independence, Expectation, Variance

$$\mathbb{E}[X] = \sum_x x\, p_X(x), \qquad \mathbb{E}[g(X)] = \sum_x g(x)\, p_X(x) \quad \text{(LOTUS)}.$$

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Standard deviation $\sigma_X = \sqrt{\mathrm{Var}(X)}$.

For constants a, b and random variables X, Y:

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b, \qquad \mathrm{Var}(aX + b) = a^2\mathrm{Var}(X).$$

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y] \quad \text{(always)}.$$

If $X \perp\!\!\!\perp Y$:

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y], \qquad \mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y).$$

Appendix*: Moment Generating Functions (non-exam background)

The MGF $M_X(t) = \mathbb{E}[e^{tX}]$ generates moments via $\mathbb{E}[X^k] = M_X^{(k)}(0)$, and determines the distribution uniquely. For independent X, Y: $M_{X+Y}(t) = M_X(t)M_Y(t)$. This underlies the proof of the CLT (Ch. 6).

4 Continuous Random Variables (Ch. 4)

A continuous RV X has a **probability density function (PDF)** $f_X(x) \geq 0$ with $\int_{-\infty}^{\infty} f_X(x)\,dx = 1$ and

$$F_X(x) = P(X \leq x) = \int_{-\infty}^{x} f_X(t)\,dt, \qquad f_X(x) = F_X'(x).$$

$P(X = x) = 0$ for any single point; $P(a \leq X \leq b) = F_X(b) - F_X(a) = \int_a^b f_X(x)\,dx$.

$$\mathbb{E}[X] = \int x f_X(x)\,dx, \qquad \mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \int (x - \mathbb{E}[X])^2 f_X(x)\,dx.$$

4.1 The Uniform Distribution

$X \sim \mathrm{Unif}(a,b)$: $f_X(x) = \frac{1}{b-a}$ on $[a,b]$, $\mathbb{E}[X] = \frac{a+b}{2}$, $\mathrm{Var}(X) = \frac{(b-a)^2}{12}$.

4.2 Common Continuous Distributions

Dist.	**PDF** $f(x)$	**Support**	$\mathbb{E}[X]$	$\mathrm{Var}(X)$
$\mathcal{N}(\mu,\sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	$\mathbb{R}$	μ	σ^2
$\mathrm{Exp}(\lambda)$	$\lambda e^{-\lambda x}$	$[0,\infty)$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$
$\mathrm{Gamma}(\alpha,\lambda)$	$\frac{\lambda^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\lambda x}$	$[0,\infty)$	$\frac{\alpha}{\lambda}$	$\frac{\alpha}{\lambda^2}$
$\mathrm{Beta}(\alpha,\beta)$	$\frac{1}{B(\alpha,\beta)} x^{\alpha-1}(1-x)^{\beta-1}$	$[0,1]$	$\frac{\alpha}{\alpha+\beta}$	$\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$

Standard normal: $Z = \frac{X-\mu}{\sigma} \sim \mathcal{N}(0,1)$, density $\varphi(z)$, CDF $\Phi(z)$ (read off the standard normal table). $\mathrm{Exp}(\lambda)$ is *memoryless*: $P(X > s+t \mid X > s) = P(X > t)$, and is the continuous analogue of the Geometric distribution; sums of α i.i.d. $\mathrm{Exp}(\lambda)$ give $\mathrm{Gamma}(\alpha,\lambda)$ (waiting time for the α-th event of a Poisson process).

4.3 Quantiles, Simulation, and Transformations

The α-quantile q_α of X solves $F_X(q_\alpha) = \alpha$, i.e. $q_\alpha = F_X^{-1}(\alpha)$. The median is $q_{0.5}$.

If $U \sim \mathrm{Unif}(0,1)$ and $X = F_X^{-1}(U)$, then X has CDF F_X. This is the standard recipe for simulating any continuous RV from uniform random numbers.

If $Y = g(X)$ with g strictly monotonic and differentiable,

$$f_Y(y) = f_X\big(g^{-1}(y)\big) \left| \frac{d}{dy} g^{-1}(y) \right|.$$

Example. If $X \sim \mathcal{N}(0,1)$ and $Y = \mu + \sigma X$, then $Y \sim \mathcal{N}(\mu,\sigma^2)$ — this is exactly how the general normal distribution is derived from the standard one.

5 Multidimensional Distributions (Ch. 5)

5.1 Joint Distributions

Discrete: joint PMF $p_{X,Y}(x,y) = P(X = x, Y = y)$. Marginal: $p_X(x) = \sum_y p_{X,Y}(x,y)$. Conditional: $p_{X|Y}(x \mid y) = \dfrac{p_{X,Y}(x,y)}{p_Y(y)}$.

Continuous: joint PDF $f_{X,Y}(x,y)$ with $P((X,Y) \in A) = \iint_A f_{X,Y}\,dx\,dy$. Marginal: $f_X(x) = \int f_{X,Y}(x,y)\,dy$. Conditional: $f_{X|Y}(x \mid y) = \dfrac{f_{X,Y}(x,y)}{f_Y(y)}$.

$X \perp\!\!\!\perp Y \iff f_{X,Y}(x,y) = f_X(x) f_Y(y)$ for all x, y (equivalently for PMFs).

5.2 Covariance and Correlation

$$\mathrm{Cov}(X,Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

$$\mathrm{Corr}(X,Y) = \rho_{X,Y} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1,1].$$

General variance of a sum:

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X,Y).$$

Properties: $\mathrm{Cov}(a+bX, c+dY) = bd\,\mathrm{Cov}(X,Y)$; $\mathrm{Corr}(a+bX, c+dY) = \mathrm{sign}(bd)\mathrm{Corr}(X,Y)$; $X \perp\!\!\!\perp Y \Rightarrow \mathrm{Cov}(X,Y) = 0$ (converse false in general — zero covariance only rules out *linear* dependence).

5.3 The Bivariate Normal and Mixture Distributions

The bivariate normal $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ is fully determined by the mean vector $\boldsymbol{\mu}$ and covariance matrix $\Sigma = \begin{pmatrix} \sigma_X^2 & \rho\sigma_X\sigma_Y \\ \rho\sigma_X\sigma_Y & \sigma_Y^2 \end{pmatrix}$; its contour lines are ellipses, and for jointly normal RVs, $\mathrm{Cov} = 0 \Rightarrow$ independence (special property of the normal family).

A **mixture distribution** is a convex combination of densities, $f(x) = \sum_k \pi_k f_k(x)$ with $\sum_k \pi_k = 1, \pi_k \geq 0$ — the basis for Gaussian mixture models and K-means' generative view (Ch. 24).

Part II — Statistics

6 Law of Large Numbers and Central Limit Theorem (Ch. 6)

For $X \geq 0$ and $a > 0$:

$$P(X \geq a) \leq \frac{\mathbb{E}[X]}{a} \quad \text{(Markov)}.$$

For any X with finite variance and $\varepsilon > 0$:

$$P(|X - \mathbb{E}[X]| \geq \varepsilon) \leq \frac{\mathrm{Var}(X)}{\varepsilon^2} \quad \text{(Chebyshev)}.$$

Both give distribution-free bounds on tail probabilities using only $\mathbb{E}[X]$ (and $\mathrm{Var}(X)$).

Let $X_1, \ldots, X_n$ be i.i.d. with mean μ and let $\bar{X}_n = \frac{1}{n}\sum_{i=1}^n X_i$. **Weak LLN (WLLN)**: for every $\varepsilon > 0$, $P(|\bar{X}_n - \mu| \geq \varepsilon) \to 0$ as $n \to \infty$ (convergence in probability). **Strong LLN (SLLN)**: $\bar{X}_n \to \mu$ almost surely. Interpretation: sample averages converge to the true mean as $n \to \infty$.

If $X_1, \ldots, X_n$ are i.i.d. with mean μ and variance $\sigma^2 < \infty$, then as $n \to \infty$,

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1),$$

equivalently $\bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$ and $\sum_{i=1}^n X_i \approx \mathcal{N}(n\mu, n\sigma^2)$ for large n.

The CLT is the justification for treating $\bar{X}_n$ as (approximately) normal regardless of the underlying distribution of X_i — this is exactly what makes confidence intervals (Ch. 10) and z/t-tests (Ch. 11) valid for large n even when the data are not normal. Rule of thumb: $n \geq 30$ is "large enough" unless the underlying distribution is extremely skewed.

7 Descriptive Statistics (Ch. 7)

For data $x_1, \ldots, x_n$:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \quad \text{(Bessel's correction)}.$$

Dividing by $n-1$ instead of n makes s^2 an unbiased estimator of σ^2 (Ch. 9). Standard deviation $s = \sqrt{s^2}$.

7.1 Quantiles, Boxplots, and the Empirical CDF

The empirical CDF is $\hat{F}_n(x) = \frac{1}{n}\#\{i : x_i \le x\}$, a step function approximating F_X. The sample α-quantile is (roughly) the value $\hat{q}_\alpha$ such that a fraction α of the data lies below it; the **median** is $\hat{q}_{0.5}$.

Boxplots display the median, first/third quartiles (Q_1, Q_3), the interquartile range $\text{IQR} = Q_3 - Q_1$, whiskers (typically up to $1.5 \times \text{IQR}$ beyond the box), and outliers as individual points.

Probability model	**Data / empirical analogue**
$\mathbb{E}[X]$ (true mean)	$\bar{x}$ (sample mean)
$\text{Var}(X)$ (true variance)	s^2 (sample variance)
F_X (CDF)	$\hat{F}_n$ (empirical CDF)
quantile q_α	sample quantile $\hat{q}_\alpha$

8 Introduction to Statistical Inference (Ch. 8)

1. **Point estimation**: compute a single best-guess value $\hat{\theta}$ for an unknown parameter θ (Ch. 9).
2. **Confidence intervals**: give a range of plausible values for θ with a stated confidence level (Ch. 10).
3. **Hypothesis testing**: decide between two competing claims about θ based on data (Ch. 11).

All three rest on treating the observed data as a realization of random variables drawn from a distribution governed by θ.

Q–Q plots compare sample quantiles to theoretical quantiles of a reference distribution (usually normal); points lying on the diagonal $y = x$ indicate the data are well-modeled by that distribution — a quick visual normality check used before applying t-based methods.

9 Point Estimates (Ch. 9)

An *estimator* $\hat{\theta} = \hat{\theta}(X_1, \ldots, X_n)$ is a function of the data used to estimate a parameter θ. It is **unbiased** if $\mathbb{E}[\hat{\theta}] = \theta$, and **consistent** if $\hat{\theta} \xrightarrow{P} \theta$ as $n \to \infty$. The **mean squared error** is $\mathrm{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}(\hat{\theta})^2$.

Set sample moments equal to theoretical moments and solve for the parameters: e.g. for one parameter θ, solve $\mathbb{E}_\theta[X] = \bar{x}$ for θ. Simple but not always efficient.

1. Write the likelihood $L(\theta) = \prod_{i=1}^n f_\theta(x_i)$ (or PMF).
2. Take the log-likelihood $\ell(\theta) = \sum_{i=1}^n \log f_\theta(x_i)$.
3. Differentiate w.r.t. θ, set $\ell'(\theta) = 0$, solve for $\hat{\theta}_{\mathrm{MLE}}$.
4. Check it is a maximum (second derivative < 0, or boundary check).

Examples.

- $X_i \overset{iid}{\sim} \mathrm{Poi}(\lambda)$: $\hat{\lambda}_{\mathrm{MLE}} = \bar{x}$.
- $X_i \overset{iid}{\sim} \mathcal{N}(\mu, \sigma^2)$: $\hat{\mu}_{\mathrm{MLE}} = \bar{x}$, $\hat{\sigma}^2_{\mathrm{MLE}} = \frac{1}{n}\sum_i (x_i - \bar{x})^2$ (biased; note the $1/n$ vs. $1/(n-1)$ contrast with s^2).
- $X_i \overset{iid}{\sim} \mathrm{Unif}(0, \theta)$: $\hat{\theta}_{\mathrm{MLE}} = \max_i x_i$ (boundary case — not found by setting the derivative to zero).

$\bar{X}_n$ is always unbiased and consistent for $\mu = \mathbb{E}[X]$ (by LLN). The MLE variance estimator $\frac{1}{n}\sum (x_i - \bar{x})^2$ is *biased* (slightly underestimates σ^2); s^2 with $n-1$ is the unbiased correction. Both are consistent (the bias vanishes as $n \to \infty$).

10 Confidence Intervals (Ch. 10)

A $(1-\alpha)$-confidence interval (CI) for θ is a random interval $[L, U]$ (computed from the data) such that $P(\theta \in [L, U]) = 1-\alpha$, *before* the data are observed. After observing the data, $[L, U]$ is a fixed interval that either does or does not contain θ; "95% confidence" describes the long-run coverage of the procedure, not a probability statement about the fixed realized interval.

Let $z_{\alpha/2}$ ($t_{\alpha/2,n-1}$) denote the $(1-\alpha/2)$-quantile of $\mathcal{N}(0,1)$ (resp. t_{n-1}). σ **known** (or n large, CLT applies):

$$\bar{x} \pm z_{\alpha/2}\,\frac{\sigma}{\sqrt{n}}.$$

σ **unknown** (estimate by s, use t-distribution for small n):

$$\bar{x} \pm t_{\alpha/2,\,n-1}\,\frac{s}{\sqrt{n}}.$$

Common multipliers: $z_{0.025} = 1.96$ (95%), $z_{0.005} = 2.576$ (99%).

For $\hat{p} = \frac{1}{n}\sum_i X_i$ with $X_i \overset{iid}{\sim} \text{Bern}(p)$ and n large (CLT):

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Width of a CI scales as $1/\sqrt{n}$: to halve the width, quadruple n. Increasing the confidence level (e.g. 95%→99%) widens the interval (larger $z_{\alpha/2}$). Use the t-distribution (not z) whenever σ is unknown and n is small/moderate; as $n \to \infty$, $t_{n-1} \to \mathcal{N}(0,1)$.

11 Hypothesis Testing (Ch. 11)

H_0 (null hypothesis, status quo) vs. H_1 (alternative). A test rejects or fails to reject H_0 based on data.

	H_0 true	H_0 false
Reject H_0	Type I error (prob. α)	Correct (power $= 1-\beta$)
Fail to reject H_0	Correct	Type II error (prob. β)

α = significance level (chosen in advance, e.g. 0.05); β depends on the true parameter value and n.

1. Compute a test statistic T from the data whose distribution under H_0 is known.
2. The **p-value** is $P(\,|T| \geq |t_{\text{obs}}| \mid H_0\,)$ (two-sided; one-sided analogues for one-sided H_1).
3. **Reject** H_0 iff p-value $< \alpha$ (equivalently, $|t_{\text{obs}}|$ exceeds the critical value).

$H_0 : \mu = \mu_0$. Test statistic:

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} \ (\sigma \text{ known}), \qquad t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \ (\sigma \text{ unknown}, \ t \sim t_{n-1} \text{ under } H_0).$$

A two-sided level-α test rejects $H_0 : \theta = \theta_0$ *iff* θ_0 lies outside the $(1-\alpha)$ confidence interval for θ. CIs and hypothesis tests are two views of the same inferential procedure.

Appendix*: Multiple Testing

When performing m independent tests at level α, the probability of at least one false positive (family-wise error rate) is $1-(1-\alpha)^m$, which grows quickly with m. The **Bonferroni correction** controls this by testing each hypothesis at level α/m.

"Statistically significant" ($p < \alpha$) is not the same as "practically important" — with large n, even tiny, irrelevant effects become significant. Always interpret effect size alongside the p-value/CI.

12 Comparison of Two Samples (Ch. 12)

Paired data: each observation in sample 1 corresponds naturally to one in sample 2 (e.g. before/after on the same subject) — reduce to a one-sample test on the differences $d_i = x_i - y_i$. **Independent samples**: two separate groups with no natural pairing.

$H_0 : \mu_X = \mu_Y$. For independent samples of sizes n, m with sample means $\bar{x}, \bar{y}$ and variances s_X^2, s_Y^2, the (Welch) test statistic is

$$t = \frac{\bar{x} - \bar{y}}{\sqrt{\dfrac{s_X^2}{n} + \dfrac{s_Y^2}{m}}},$$

compared against a t-distribution with (Welch–Satterthwaite) approximate degrees of freedom. If variances are assumed equal, a pooled-variance version is used instead.

Paired t-test on $d_i = x_i - y_i$ is almost always more powerful than an independent two-sample test when pairing is natural, because it removes between-subject variability that would otherwise inflate s_X^2, s_Y^2.

Part III — Introduction to Machine Learning

13 Introduction to Linear Regression (Ch. 13)

Given training data $\{(x_i, y_i)\}_{i=1}^n$ (features x_i, labels/targets y_i), supervised learning fits a model $\hat{y} = f_\theta(x)$ by choosing θ to minimize a loss on the training data, then evaluates generalization on unseen (test) data. **Regression**: y continuous. **Classification**: y categorical (Ch. 15–16).

Model: $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$. Ordinary least squares (OLS) chooses $(\hat{\beta}_0, \hat{\beta}_1)$ minimizing $\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$, giving

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{\widehat{\mathrm{Cov}}(x, y)}{\widehat{\mathrm{Var}}(x)}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

$$R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \in [0, 1].$$

Fraction of variance in y explained by the model. $R^2 = 1$: perfect fit; $R^2 = 0$: model no better than predicting $\bar{y}$ for everything.

14 Multivariate Linear Regression and Regularization (Ch. 14)

For design matrix $X \in \mathbb{R}^{n \times p}$ (rows = samples, possibly with an intercept column of 1s) and target $\boldsymbol{y} \in \mathbb{R}^n$, OLS minimizes $\|\boldsymbol{y} - X\boldsymbol{\beta}\|^2$, giving the closed-form solution

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \boldsymbol{y},$$

valid when $X^\top X$ is invertible (requires $n \geq p$ and no perfect multicollinearity).

Both add a penalty on $\boldsymbol{\beta}$ to the OLS objective to control overfitting (regularization), controlled by $\lambda \geq 0$:

$$\textbf{Ridge:} \quad \min_{\boldsymbol{\beta}} \ \|\boldsymbol{y} - X\boldsymbol{\beta}\|^2 + \lambda \|\boldsymbol{\beta}\|_2^2, \qquad \textbf{Lasso:} \quad \min_{\boldsymbol{\beta}} \ \|\boldsymbol{y} - X\boldsymbol{\beta}\|^2 + \lambda \|\boldsymbol{\beta}\|_1.$$

Ridge has the closed form $\hat{\boldsymbol{\beta}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \boldsymbol{y}$ (always invertible for $\lambda > 0$) and shrinks coefficients smoothly toward 0. Lasso's ℓ_1 penalty induces *sparsity* (some $\hat{\beta}_j$ exactly 0) — automatic feature selection. Larger $\lambda \Rightarrow$ more shrinkage $\Rightarrow$ higher bias, lower variance.

14.1 Non-Linear Features: Kernels and the Curse of Dimensionality

Non-linear relationships can be captured by augmenting X with non-linear feature transformations (polynomial features, or a **radial basis function (RBF) kernel** $k(x, x') = \exp(-\|x - x'\|^2/(2\ell^2))$), then applying linear regression in the transformed space — the basis of **support vector regression (SVR)** and kernel ridge regression.

Curse of dimensionality: as the number of features p grows, the volume of the feature space grows exponentially, so data become sparse and "nearest neighbors" become distant — distance-based methods (kernels, trees, k-NN, GPs) degrade, and overfitting risk increases unless n grows correspondingly or regularization/dimensionality reduction (Part VI) is used.

15 Classification Trees (Ch. 15)

A decision tree partitions the feature space recursively via axis-aligned splits (e.g. $x_j \leq c$), forming a binary tree. Each leaf is assigned a predicted class (majority class of training points in that leaf) or, for regression, a predicted value (e.g. mean of training targets in the leaf).

Trees are simple to interpret and naturally handle mixed feature types and non-linear decision boundaries, but a single deep tree tends to **overfit** (Ch. 16, Part IV addresses this via ensembles).

16 Training Classification Trees (Ch. 16)

For a node with class proportions $p_1, \dots, p_K$, the **entropy** is

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k \quad (0 \log_2 0 := 0).$$

$H = 0$: pure node (single class); H maximal ($= \log_2 K$) when classes are equally likely. A split into children with weighted entropy $H_{\text{children}} = \sum_j \frac{n_j}{n} H_j$ has

$$\text{Information Gain} = H_{\text{parent}} - H_{\text{children}}.$$

(**Gini impurity** $G = 1 - \sum_k p_k^2$ is a common alternative to entropy.)

At each node: for every feature j and candidate threshold c, compute the information gain of splitting on $x_j \le c$; choose the (j, c) that maximizes information gain; recurse on the two children. Stop when a node is pure, reaches a minimum size, or a maximum depth — these *stopping criteria* (or post-hoc *pruning*) control overfitting.

Part IV — Ensembles and Foundations of Learning

17 Ensemble Methods

Bagging (bootstrap aggregating): train B models on independent bootstrap samples (sampling n points with replacement from the training set) and average their predictions (regression) or take a majority vote (classification). **Random Forest**: bagging of decision trees where, additionally, each split considers only a random subset of features — decorrelates the trees further.

If B predictors each have variance σ^2 and pairwise correlation ρ, the variance of their average is

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x)\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

As $B \to \infty$, this $\to \rho\sigma^2$: averaging reduces variance, but residual correlation ρ between predictors caps the benefit — motivating Random Forest's extra feature randomization to reduce ρ.

Boosting (e.g. AdaBoost, Gradient Boosting) builds an ensemble *sequentially*: each new model focuses on the errors (residuals, or reweighted misclassified points) of the current ensemble, and models are combined with weights. Unlike bagging (parallel, variance reduction), boosting primarily reduces **bias**, but can overfit if the number of rounds is too large.

	Bagging / Random Forest	**Boosting**
Training	Parallel, independent models	Sequential, dependent models
Reduces	Variance	Bias (and variance)
Base learners	Deep/unpruned trees	Shallow trees ("stumps")
Overfitting risk	Low (more trees $\rightarrow$ better)	Higher (too many rounds overfits)

18 Foundations of Inference and Learning

Frequentist: parameters θ are fixed unknown constants; probability describes the sampling distribution of estimators/data (CIs, p-values as in Ch. 10–11). **Bayesian**: θ is itself a random variable with a **prior** $p(\theta)$; observing data D updates this to a **posterior** via Bayes' theorem,

$$p(\theta \mid D) \propto p(D \mid \theta)\, p(\theta).$$

$$\hat{\theta}_{\text{MLE}} = \arg\max_\theta p(D \mid \theta), \qquad \hat{\theta}_{\text{MAP}} = \arg\max_\theta p(D \mid \theta)\, p(\theta) = \arg\max_\theta \big[\, \log p(D \mid \theta) + \log p(\theta) \,\big].$$

MAP = MLE + a prior-induced penalty term; with a flat (uninformative) prior, MAP = MLE. A Gaussian prior on θ corresponds to an ℓ_2 (Ridge-type) penalty; a Laplace prior corresponds to an ℓ_1 (Lasso-type) penalty.

For a model $\hat{f}$ predicting $y = f(x) + \varepsilon$ (with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$), the expected test error at a point x decomposes as

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f(x)\big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible error}}.$$

Bias–variance tradeoff: increasing model complexity (more features, deeper trees, smaller λ) typically *decreases* bias but *increases* variance, and vice versa. Regularization (Ridge/Lasso, Ch. 14), bagging (Part IV), and choosing λ/depth via cross-validation all navigate this tradeoff. Total test error is minimized at an intermediate complexity — not at the most flexible model.

19 Gaussian Processes

A Gaussian Process (GP) is a distribution over functions $f : \mathcal{X} \to \mathbb{R}$ such that, for any finite set of inputs $x_1, \dots, x_n$, the vector $(f(x_1), \dots, f(x_n))$ is jointly Gaussian. A GP is fully specified by a mean function $m(x)$ (often 0) and a covariance (**kernel**) function $k(x, x')$.

Given noisy observations $y_i = f(x_i) + \varepsilon_i$, $\varepsilon_i \overset{iid}{\sim} \mathcal{N}(0, \sigma_n^2)$, at training inputs X with kernel matrix $K = [k(x_i, x_j)]$, the posterior predictive distribution at a new point x_* is Gaussian with

$$\mathbb{E}[f(x_*) \mid D] = \boldsymbol{k}_*^\top (K + \sigma_n^2 I)^{-1} \boldsymbol{y}, \qquad \mathrm{Var}[f(x_*) \mid D] = k(x_*, x_*) - \boldsymbol{k}_*^\top (K + \sigma_n^2 I)^{-1} \boldsymbol{k}_*,$$

where $\boldsymbol{k}_* = [k(x_*, x_i)]_{i=1}^n$. The posterior mean is the prediction; the posterior variance quantifies **uncertainty** — larger far from observed data.

RBF (squared-exponential) kernel: $k(x, x') = \sigma_f^2 \exp\left(-\dfrac{\|x - x'\|^2}{2\ell^2}\right)$, with hyperparameters: signal variance σ_f^2 (output scale), length scale ℓ (how fast correlation decays with distance — controls smoothness), and noise variance σ_n^2. Hyperparameters are typically chosen by maximizing the marginal likelihood.

GPs generalize Bayesian linear regression to function space and provide **calibrated uncertainty estimates** (the posterior variance), unlike a single point-estimate model (e.g. plain OLS) — valuable for active learning / Bayesian optimization where you decide *where* to sample next based on predicted uncertainty.

Part V — Deep Learning

20 Neural Networks

A feedforward network computes, layer by layer,

$$\boldsymbol{h}^{(l)} = \sigma\big(W^{(l)} \boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big), \qquad \boldsymbol{h}^{(0)} = \boldsymbol{x},$$

where $W^{(l)}, \boldsymbol{b}^{(l)}$ are the weights/biases of layer l and σ is a non-linear **activation function** (e.g. ReLU $\sigma(z) = \max(0, z)$, sigmoid, tanh). Without a non-linear σ, stacking layers collapses to a single linear map.

Training minimizes a loss $\mathcal{L}(\theta)$ (e.g. MSE for regression, cross-entropy for classification) over network parameters $\theta = \{W^{(l)}, \boldsymbol{b}^{(l)}\}$. **Backpropagation** computes $\nabla_\theta \mathcal{L}$ efficiently via the chain rule, propagating gradients backward from the output layer. **Stochastic gradient descent (SGD)** updates

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}_{\text{batch}}(\theta),$$

where η is the **learning rate** and $\mathcal{L}_{\text{batch}}$ is estimated on a mini-batch rather than the full dataset (faster, noisier updates that can help escape poor local minima).

Vanishing/exploding gradients: in deep networks, repeated multiplication of Jacobians during backpropagation can make gradients shrink toward 0 (vanishing — early layers stop learning) or grow unboundedly (exploding). Sigmoid/tanh activations saturate and worsen vanishing gradi-

ents; ReLU, careful weight initialization, batch normalization, and residual ("skip") connections mitigate this. Learning rate too large ⇒ divergence; too small ⇒ extremely slow convergence.

21 Convolutional Neural Networks (CNNs)

A convolutional layer slides a small learnable **filter** (kernel) of weights across the input (e.g. an image), computing a weighted sum (dot product) at each position to produce a **feature map**. Key ideas:

- **Parameter sharing**: the same filter weights are reused at every spatial location ⇒ far fewer parameters than a fully connected layer, and translation equivariance.
- **Local receptive field**: each output unit depends only on a small local patch of the input.
- **Stride** and **padding** control the output size; multiple filters per layer produce multiple feature maps (channels).

Pooling (e.g. max-pooling, average-pooling) downsamples feature maps by summarizing small spatial regions (e.g. 2×2) with a single value, reducing spatial resolution and parameter count in subsequent layers while adding a degree of translation invariance. A typical CNN alternates convolution + activation + pooling, then ends with fully connected layers for the final prediction.

22 NLP and Transformers

An **embedding** maps discrete tokens (words/subwords) to dense, low-dimensional vectors $\boldsymbol{e} \in \mathbb{R}^d$ learned such that semantically similar tokens have similar vectors. Embeddings are the input representation for downstream sequence models (RNNs, Transformers).

For input embeddings packed into matrices of queries Q, keys K, values V (each a learned linear projection of the input), **scaled dot-product attention** is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where d_k is the key dimension. Each output is a weighted combination of all value vectors, with weights determined by the similarity (dot product) between queries and keys — this lets every position attend to every other position directly, regardless of distance. **Multi-head attention** runs several attention operations in parallel on different learned projections and concatenates the results.

A Transformer block consists of a multi-head self-attention layer followed by a position-wise feedforward network, with **residual connections** and **layer normalization** around each sub-layer. Since attention itself is permutation-invariant, **positional encodings** are added to the input embeddings to inject order information. Encoder-only (e.g. BERT), decoder-only (e.g. GPT, autoregressive/causal masking), and encoder–decoder variants exist.

Self-attention's computational and memory cost scales as $O(n^2)$ in the sequence length n (every token attends to every other token) — the key practical bottleneck for long sequences, motivating efficient-attention variants. Unlike RNNs, Transformers process all positions in parallel (faster training) but need positional encodings since they have no inherent notion of order.

Part VI — Unsupervised Learning

23 Clustering

Given unlabeled data $x_1, \ldots, x_n$, clustering partitions the points into K groups such that points within a group are similar and points in different groups are dissimilar, according to some notion of distance/similarity.

Minimize the within-cluster sum of squares

$$J = \sum_{k=1}^{K} \sum_{i:\, z_i = k} \|x_i - \mu_k\|^2$$

over cluster assignments $z_i \in \{1, \ldots, K\}$ and centroids μ_k. Lloyd's algorithm alternates:

1. **Assign**: $z_i \leftarrow \arg\min_k \|x_i - \mu_k\|^2$ for each i.
2. **Update**: $\mu_k \leftarrow$ mean of $\{x_i : z_i = k\}$.

Repeat until convergence (assignments stop changing). J is non-increasing at each step but the algorithm only finds a **local** optimum — run with multiple random initializations (e.g. k-means++) and pick the best result.

A GMM models the data as a mixture $p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$. The **Expectation-Maximization (EM)** algorithm fits π_k, μ_k, Σ_k by alternating:

- **E-step**: compute the responsibilities $r_{ik} = P(z_i = k \mid x_i)$ (soft cluster assignments) using the current parameters.
- **M-step**: update π_k, μ_k, Σ_k using the responsibilities as soft weights (weighted means/covariances).

K-means is a special case of EM/GMM with hard assignments and shared isotropic covariance.

K-means uses Euclidean distance, so features on different scales dominate the distance calculation — always **standardize/normalize** features before clustering. K-means assumes roughly spherical, equal-sized clusters; GMMs are more flexible (elliptical clusters via Σ_k, soft assignments, and a proper probabilistic model with a log-likelihood for model comparison).

24 Dimensionality Reduction

PCA finds an orthogonal linear transformation to new axes (**principal components**, PCs) that are ordered by how much variance of the (centered) data they explain. The PCs are the eigenvectors of the data covariance matrix Σ, with the corresponding eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$ giving the variance along each PC.

The fraction of total variance explained by the first m principal components is

$$\frac{\sum_{j=1}^{m} \lambda_j}{\sum_{j=1}^{p} \lambda_j},$$

where p is the original dimensionality. Projecting onto the top m PCs gives the best (in squared-error sense) m-dimensional linear approximation of the data — used for visualization, noise reduction, and as preprocessing before other ML methods (mitigating the curse of dimensionality).

t-SNE is a non-linear dimensionality reduction technique designed for *visualization* (typically to 2D/3D): it preserves local neighborhood structure (similar points stay close) by matching pairwise similarity distributions in high- and low-dimensional space, at the cost of distorting global distances/-sizes of clusters.

PCA vs. t-SNE: PCA is linear, deterministic, preserves global variance structure, and its axes are interpretable (directions of maximal variance) — suitable as a preprocessing step feeding into downstream models. t-SNE is non-linear, stochastic (depends on random initialization/seed), good for visual cluster discovery but *distances between clusters and cluster sizes in a t-SNE plot are not meaningful* and it should not be used as a general preprocessing step.

Part VII — Generative Models

25 Autoencoders and Variational Autoencoders

An autoencoder is a neural network trained to reconstruct its input: an **encoder** $z = g_\phi(x)$ maps the input to a lower-dimensional **latent code** z (the *bottleneck*), and a **decoder** $\hat{x} = h_\theta(z)$ reconstructs the input from z. Trained by minimizing reconstruction error $\|x - \hat{x}\|^2$. The bottleneck forces the network to learn a compressed representation capturing the most salient structure of the data.

A VAE treats the latent code as random, $z \sim q_\phi(z \mid x)$ (typically $\mathcal{N}(\mu_\phi(x), \Sigma_\phi(x))$), with a prior $p(z) = \mathcal{N}(0, I)$, and a decoder $p_\theta(x \mid z)$. Training maximizes the **evidence lower bound (ELBO)**:

$$\text{ELBO} = \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{regularization term}}.$$

The KL term pulls the latent distribution toward the prior $\mathcal{N}(0, I)$, giving a smooth, continuous latent space from which new samples can be generated by drawing $z \sim p(z)$ and decoding.

26 Generative Adversarial Networks (GANs)

A GAN consists of a **generator** G, mapping noise $z \sim p(z)$ to synthetic samples $G(z)$, and a **discriminator** D, trained to distinguish real samples x from generated samples $G(z)$. The two are trained adversarially.

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))].$$

D is trained to maximize this (correctly classify real vs. fake); G is trained to minimize it (fool D). At the (theoretical) equilibrium, G produces samples indistinguishable from real data and $D(x) = \frac{1}{2}$ everywhere.

Mode collapse: the generator may learn to produce only a small variety of outputs (e.g. one or a few "modes" of the data distribution) that reliably fool D, rather than capturing the full diversity of p_{data}. GAN training is also notoriously unstable (the minimax objective has no guarantee of convergence to equilibrium with simultaneous gradient updates) — common practical fixes include alternative loss formulations, gradient penalties, and careful balancing of G/D training.

27 Diffusion Models

A diffusion model defines a fixed forward process that gradually adds Gaussian noise to data $x_0 \sim p_{\text{data}}$ over T steps:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, I),$$

so that x_T is approximately pure Gaussian noise for large T.

A neural network $\varepsilon_\theta(x_t, t)$ is trained to predict the noise added at step t, by minimizing

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0,t,\varepsilon}\big[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\big].$$

To **generate** a new sample, start from $x_T \sim \mathcal{N}(0, I)$ and iteratively apply the learned reverse (denoising) step from $t = T$ down to $t = 0$, gradually transforming noise into a sample from (approximately) p_{data}.

Diffusion models typically produce higher-quality and more diverse samples than GANs and have a more stable training objective (simple regression on noise, no adversarial min-max), but **sampling is slow** — generation requires many sequential denoising steps (often hundreds to thousands), unlike GANs/VAEs/autoencoders which generate in a single forward pass. Much recent work focuses on reducing the number of required sampling steps.

Part VIII — Reinforcement Learning

28 Markov Decision Processes and Bellman Equations

An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: states $\mathcal{S}$, actions $\mathcal{A}$, transition probabilities $P(s' \mid s, a)$, reward function $R(s, a, s')$ (or $R(s, a)$), and discount factor $\gamma \in [0, 1)$. The **Markov property**: $P(s_{t+1} \mid s_t, a_t)$ does not depend on the history before s_t. A **policy** $\pi(a \mid s)$ specifies the agent's behavior.

The **state-value function** under policy π is the expected discounted return from s:

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s\right].$$

The **action-value (Q) function** additionally fixes the first action:

$$Q^\pi(s, a) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s, a_0 = a\right].$$

Bellman expectation equation (for a fixed policy π):

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\big[R(s, a, s') + \gamma V^\pi(s')\big].$$

Bellman optimality equation (for the optimal value function $V^* = \max_\pi V^\pi$):

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a)\big[R(s, a, s') + \gamma V^*(s')\big], \qquad \pi^*(s) = \arg\max_a Q^*(s, a).$$

These recursive identities — "value of a state = immediate reward + discounted value of the next state" — underlie all of dynamic programming and RL.

29 Multi-Armed Bandits

A k-armed bandit has k actions ("arms"), each with an unknown reward distribution with mean μ_a. The agent repeatedly chooses an arm and observes a reward, aiming to maximize cumulative reward (equivalently minimize **regret** relative to always playing the best arm). The core tension is **exploration** (try arms to learn their values) vs. **exploitation** (play the currently-best-known arm).

ε-greedy: with probability $1-\varepsilon$ play $\arg\max_a \hat{\mu}_a$ (exploit); with probability ε play a uniformly random arm (explore).

Upper Confidence Bound (UCB): play

$$\arg\max_a \left[\hat{\mu}_a + c\sqrt{\frac{\ln t}{N_a}}\right],$$

where N_a is the number of times arm a has been played and t is the current round — arms with few pulls get an "optimism bonus", automatically balancing exploration and exploitation without a tunable ε.

30 Q-Learning, Policy Gradients, and PPO

Q-learning learns Q^* directly via the update (after observing s, a, r, s'):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big],$$

where α is the learning rate. The target uses $\max_{a'} Q(s',a')$ regardless of which action is actually taken next — **off-policy**: it learns about the optimal policy while following a (possibly exploratory) behavior policy.

SARSA updates using the action a' actually selected by the current policy in state s':

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma Q(s',a') - Q(s,a)\Big].$$

On-policy: it evaluates and improves the same policy that generates the behavior, making it more sensitive to the exploration strategy (e.g. ε-greedy) than Q-learning.

Policy gradient methods directly parameterize the policy $\pi_\theta(a \mid s)$ and optimize $J(\theta) = \mathbb{E}_{\pi_\theta}[\text{return}]$ via gradient ascent, using the **REINFORCE** estimator

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^\pi(s,a)\big].$$

Proximal Policy Optimization (PPO) improves stability by clipping the policy update so the new policy does not move too far from the old one in a single step, using a clipped surrogate objective on the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$.

On-policy vs. off-policy: off-policy methods (Q-learning, and methods using replay buffers) can reuse past experience generated by old/different policies, generally more sample-efficient; on-policy methods (SARSA, vanilla policy gradient, PPO) must use fresh data from the current policy, but tend to be more stable. PPO is on-policy but reuses each batch of data for several gradient steps via the clipped objective, balancing sample efficiency and stability.

Exam Quick Reference

Discrete Distributions

Dist.	PMF	Support	$\mathbb{E}[X]$	$\mathrm{Var}(X)$
$\mathrm{Unif}\{1,\dots,n\}$	$1/n$	$1,\dots,n$	$\frac{n+1}{2}$	$\frac{n^2-1}{12}$
$\mathrm{Bern}(p)$	$p^x(1-p)^{1-x}$	$\{0,1\}$	p	$p(1-p)$
$\mathrm{Bin}(n,p)$	$\binom{n}{x}p^x(1-p)^{n-x}$	$0,\dots,n$	np	$np(1-p)$
$\mathrm{Geom}(p)$	$(1-p)^{x-1}p$	$1,2,\dots$	$1/p$	$(1-p)/p^2$
$\mathrm{NegBin}(r,p)$	$\binom{x-1}{r-1}p^r(1-p)^{x-r}$	$r,r+1,\dots$	r/p	$r(1-p)/p^2$
$\mathrm{Poi}(\lambda)$	$e^{-\lambda}\lambda^x/x!$	$0,1,2,\dots$	λ	λ

Continuous Distributions

Dist.	PDF	Support	$\mathbb{E}[X]$	$\mathrm{Var}(X)$
$\mathrm{Unif}(a,b)$	$\frac{1}{b-a}$	$[a,b]$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
$\mathcal{N}(\mu,\sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/2\sigma^2}$	$\mathbb{R}$	μ	σ^2
$\mathrm{Exp}(\lambda)$	$\lambda e^{-\lambda x}$	$[0,\infty)$	$1/\lambda$	$1/\lambda^2$
$\mathrm{Gamma}(\alpha,\lambda)$	$\frac{\lambda^\alpha}{\Gamma(\alpha)}x^{\alpha-1}e^{-\lambda x}$	$[0,\infty)$	α/λ	α/λ^2
$\mathrm{Beta}(\alpha,\beta)$	$\frac{1}{B(\alpha,\beta)}x^{\alpha-1}(1-x)^{\beta-1}$	$[0,1]$	$\frac{\alpha}{\alpha+\beta}$	$\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$

Core Formulas

Probability

$$P(A\cup B)=P(A)+P(B)-P(A\cap B)$$

$$P(A\mid B)=\frac{P(A\cap B)}{P(B)}$$

$$P(B_i\mid A)=\frac{P(A\mid B_i)P(B_i)}{\sum_j P(A\mid B_j)P(B_j)}$$

Expectation / Variance / Covariance

$$\mathbb{E}[aX+b]=a\mathbb{E}[X]+b$$

$$\mathrm{Var}(aX+b)=a^2\mathrm{Var}(X)$$

$$\mathrm{Var}(X+Y)=\mathrm{Var}(X)+\mathrm{Var}(Y)+2\mathrm{Cov}(X,Y)$$

$$\mathrm{Cov}(X,Y)=\mathbb{E}[XY]-\mathbb{E}[X]\mathbb{E}[Y]$$

$$\mathrm{Corr}(X,Y)=\frac{\mathrm{Cov}(X,Y)}{\sigma_X\sigma_Y}$$

LLN / CLT / Inequalities

$$P(X\geq a)\leq\frac{\mathbb{E}[X]}{a}\quad(X\geq 0)$$

$$P(|X-\mathbb{E}[X]|\geq\varepsilon)\leq\frac{\mathrm{Var}(X)}{\varepsilon^2}$$

$$\bar{X}_n\approx\mathcal{N}\left(\mu,\frac{\sigma^2}{n}\right)\ \text{(large } n\text{)}$$

Confidence Intervals

$$\bar{x}\pm z_{\alpha/2}\frac{\sigma}{\sqrt{n}}\quad(\sigma\text{ known})$$

$$\bar{x}\pm t_{\alpha/2,n-1}\frac{s}{\sqrt{n}}\quad(\sigma\text{ unknown})$$

$$\hat{p}\pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Hypothesis Testing

$$z=\frac{\bar{x}-\mu_0}{\sigma/\sqrt{n}},\qquad t=\frac{\bar{x}-\mu_0}{s/\sqrt{n}}$$

Reject H_0 iff p-value $<\alpha$.

Regression and ML

$$\hat{\beta}_1 = \frac{\widehat{\mathrm{Cov}}(x,y)}{\widehat{\mathrm{Var}}(x)}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \boldsymbol{y}$$

$$\hat{\boldsymbol{\beta}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \boldsymbol{y}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$$\text{Bias}^2 + \text{Variance} + \sigma^2 = \text{Exp. test error}$$

$$\text{IG} = H_{\text{parent}} - \sum_j \frac{n_j}{n} H_j, \quad H = -\sum_k p_k \log_2 p_k$$

Gaussian Process Posterior

$$\mathbb{E}[f(x_*) \mid D] = \boldsymbol{k}_*^\top (K + \sigma_n^2 I)^{-1} \boldsymbol{y}$$

$$\mathrm{Var}[f(x_*) \mid D] = k(x_*, x_*) - \boldsymbol{k}_*^\top (K + \sigma_n^2 I)^{-1} \boldsymbol{k}_*$$

$$k_{\text{RBF}}(x, x') = \sigma_f^2 \exp\!\left(-\frac{\|x - x'\|^2}{2\ell^2} \right)$$

Reinforcement Learning

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a)[R + \gamma V^*(s')]$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

$$\nabla_\theta J(\theta) = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a \mid s) Q^\pi(s,a)\big]$$

Standard Normal Table — Selected Quantiles

α	0.20	0.10	0.05	0.01
z_α (one-sided)	0.8416	1.2816	1.6449	2.3263
$z_{\alpha/2}$ (two-sided)	1.2816	1.6449	1.9600	2.5758

$\Phi(z)$ is the CDF of $\mathcal{N}(0,1)$; $\Phi(-z) = 1 - \Phi(z)$. E.g. $\Phi(1.96) \approx 0.975$, so a 95% two-sided CI uses $z_{0.025} = 1.96$.